表 7.6: 从 Hub 端口到可用键盘的请求序列

对象	请求	bmRequestType	作用
Hub	SET_FEATURE(PORT_POWER)	0x23	给下游端口上电，之后最多等 500 ms 连接去抖
Hub	GET_STATUS	0xa3	端口状态：bit 0 连接、bit 1 使能、bit 8 供电、bit 9 低速、bit 10 高速
Hub	SET_FEATURE(PORT_RESET)	0x23	复位端口，设备进入默认状态、地址为 0
设备 0	GET_DESCRIPTOR(Device, 8)	0x80	只取 8 字节，得到端点 0 的最大包长
设备 0	GET_DESCRIPTOR(Device, 18)	0x80	完整设备描述符：class、VID、PID
设备 0	SET_ADDRESS(n)	0x00	分配唯一地址；把父 Hub、端口、速度记入 usb_route[n]
设备 *n*	GET_DESCRIPTOR(Config, 9)	0x80	得到 wTotalLength
设备 *n*	GET_DESCRIPTOR(Config, total)	0x80	全部接口和端点，挑出 Boot Keyboard
设备 *n*	SET_CONFIGURATION	0x00	激活配置，端点开始可用
接口	SET_PROTOCOL(boot)	0x21	让键盘用固定的 8 字节 Boot 报告格式
接口	SET_IDLE(0)	0x21	只在按键状态变化时才回报告

两层，防止一个异常的拓扑让启动无限递归。扫描到的端点属性按 USB 地址记录在 usb_ep0_mps[] 里，Hub、低速键盘和鼠标不再共用一个全局的端点 0 包长。

把这一整套判断画成控制流就是图 7.9。它有五个阶段，值得注意的不是步骤本身，而是三处分支*判据*：class 9 决定要不要扫描下游端口，端口状态决定这个端口是否继续，class/subclass/protocol 决定这个接口是不是 Boot Keyboard。递归那一跳（子 Hub 回到端口扫描）和低速那一支（LSPDDEV 加 split 事务）是真机上最容易出错的两处——前者决定嵌套 Hub 能不能被发现，后者决定 EP0 的每个包要不要单独走一遍 SSPLIT/CSPLIT。

SET_PROTOCOL(boot) 很关键。HID 设备本来用“报告描述符”描述自己的报告格式，解析它需要一个不小的解释器；而 Boot 协议是 BIOS 时代为了不解析报告描述符而规定的固定格式，所有键盘都必须支持。用它，驱动只需理解图 7.10 这 8 个字节。

最后，usb_device_register() 把设备交给 USB 总线，总线按 class/subclass/protocol 匹配到 usbhid-keyboard 驱动，调用 usbkbd_probe()：发送上面两个 HID 请求，分配每设备的 usbkbd_state 和 URB，注册 input_dev，然后第一次提交 URB。

枚举与配置的软件函数调用链. 图 7.11回答的是“这些报文由哪一个 C 函数发起、设备何时进入通用设备模型，以及键盘的 interrupt-IN 请求最终在哪里启动”；它与 §7.4.11 热插拔一节的总线时序图（图 7.18）互为表里——那张图讲“USB 线上先后发送了什么”，这张图讲“这些报文由谁发起”。图中同一行的浅灰等宽字是这一步对应的串口日志，把函数调用与真机实际输出逐行对齐。启动代码先注册 USB 总线与类驱动，随后注册 DWC2 platform 设备和驱动；因此 dwc2_probe() 发现并注册 USB 设备时，device_register() 可以立即找到已经存在的 usbkbd_driver 并完成绑定。

调用链中有三个容易混淆的边界：

1. **USB 设备地址与 DWC2 channel 不是同一个编号。** SET_ADDRESS(4) 改变的是总线上的设备地址；ch0、ch6、ch7 是 DWC2 内部的事务执行上下文。dwc2_submit_urb() 把 URB 中的 device address、endpoint、方向、速度和 split route 写入 ch6 寄存器，才建立“ch6 正在服务 addr 4 的 EP1-IN”这一临时关系。
2. **枚举和运行采用不同执行模型。** control() 在启动阶段同步等待 ch0 完成，便于严格保持 USB 枚举顺序；键盘运行期则由 ch6 IRQ 完成 URB，再由 workqueue 解析 HID report，NAK 重试通过 dwc2_periodic_wq 的 delayed work 按到期时间重提。
3. **注册设备不是重新枚举。** usb_device_register() 只把已完成地址和配置阶段的 usb_child 交给设备模型；usb_match() 选择驱动，usbkbd_probe() 完成 HID 类请求、输入设备注册和第一只运行期 URB 的提交。